# The Decorator Pattern

Human Computer Interaction Research
University of Nevada, Reno

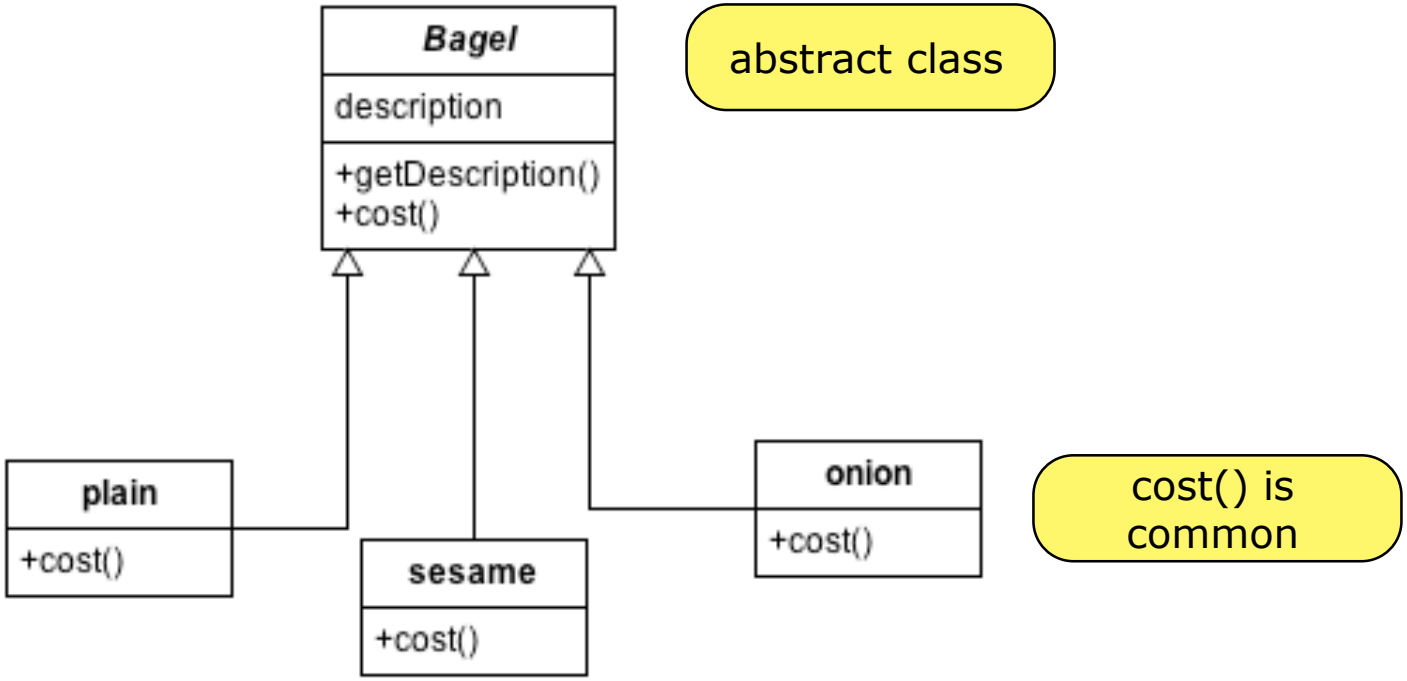
# Pattern Categories

- Behavioral Patterns
  - » observer
  - » strategy
- Structural Patterns
  - » decorator

# Problem

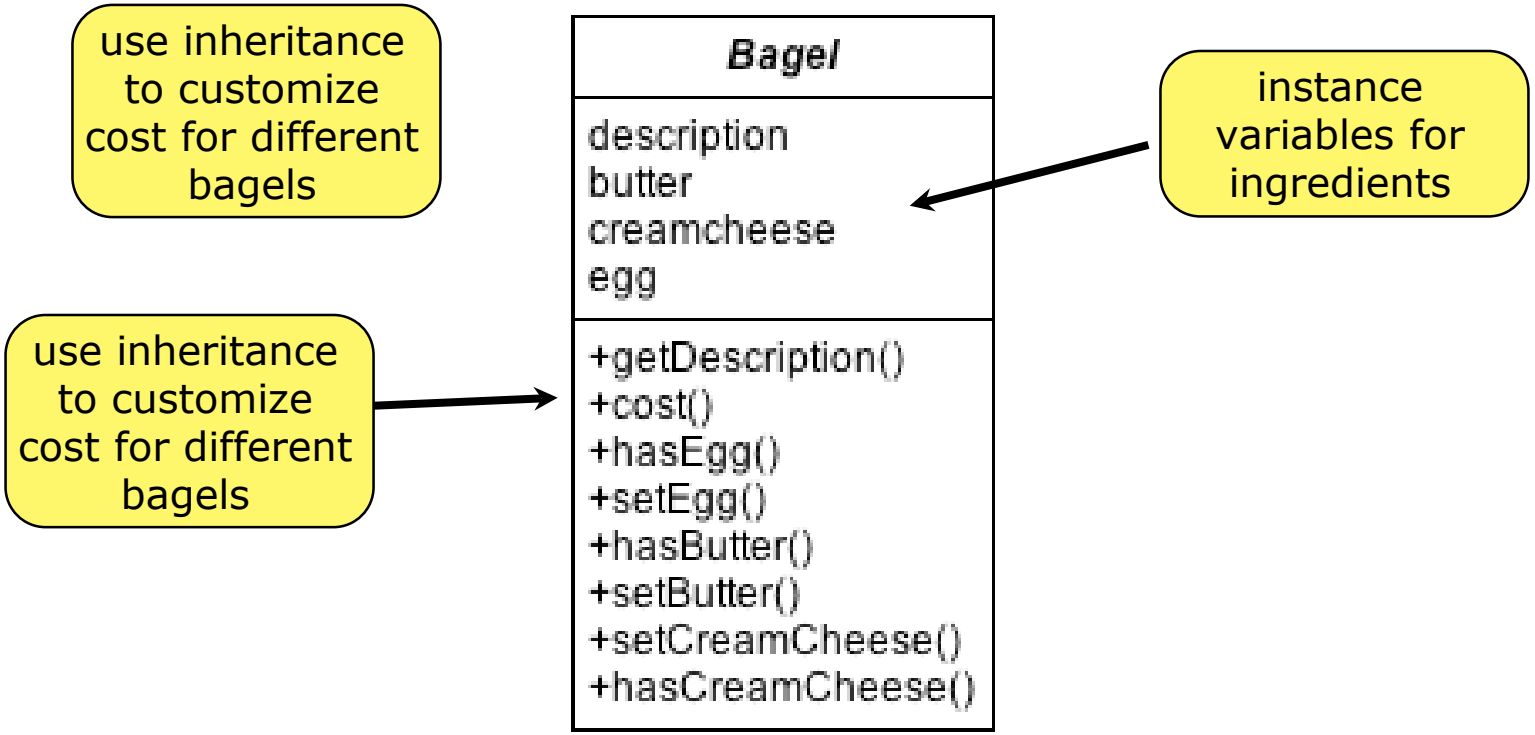
"Overuse of inheritance often leads to an explosion of classes"

# Example: Bagel Store



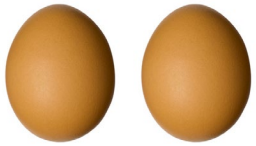

# Class explosion


# Alternative Design

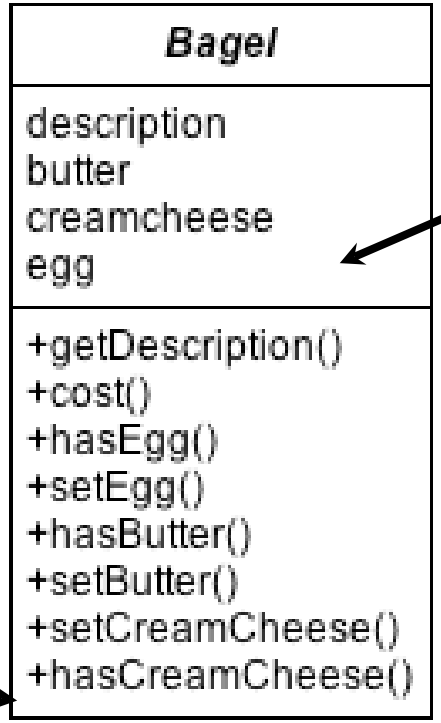


# Disadvantages

cheese?



2 eggs?




hascheese()?

# Open-closed Principle

"Classes should be open for extension, but closed for modification"

# What this means

- You can extend functionality through inheritance but that does not always achieve flexibility in our designs.
- We should be able to extend behavior without modifying existing code.
- You can use composition and delegation to add new behavior at runtime.

"Favor composition over inheritance"

# Decorator Pattern

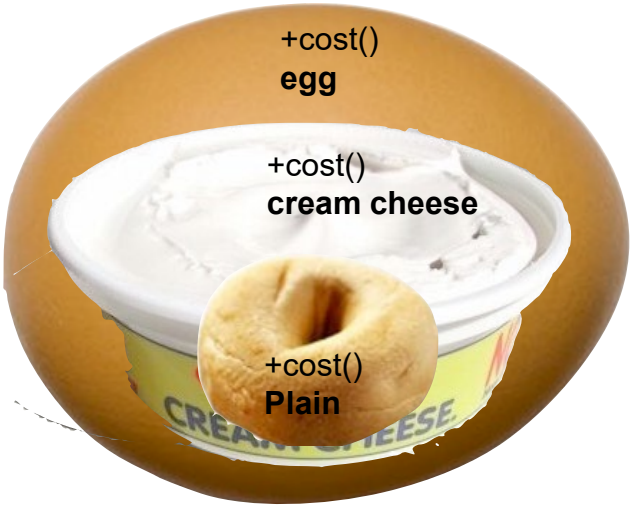
The Decorator Pattern attaches additional responsibilities to an object dynamically.

Decorators provide a flexible alternative to sub-classing for extending functionality.

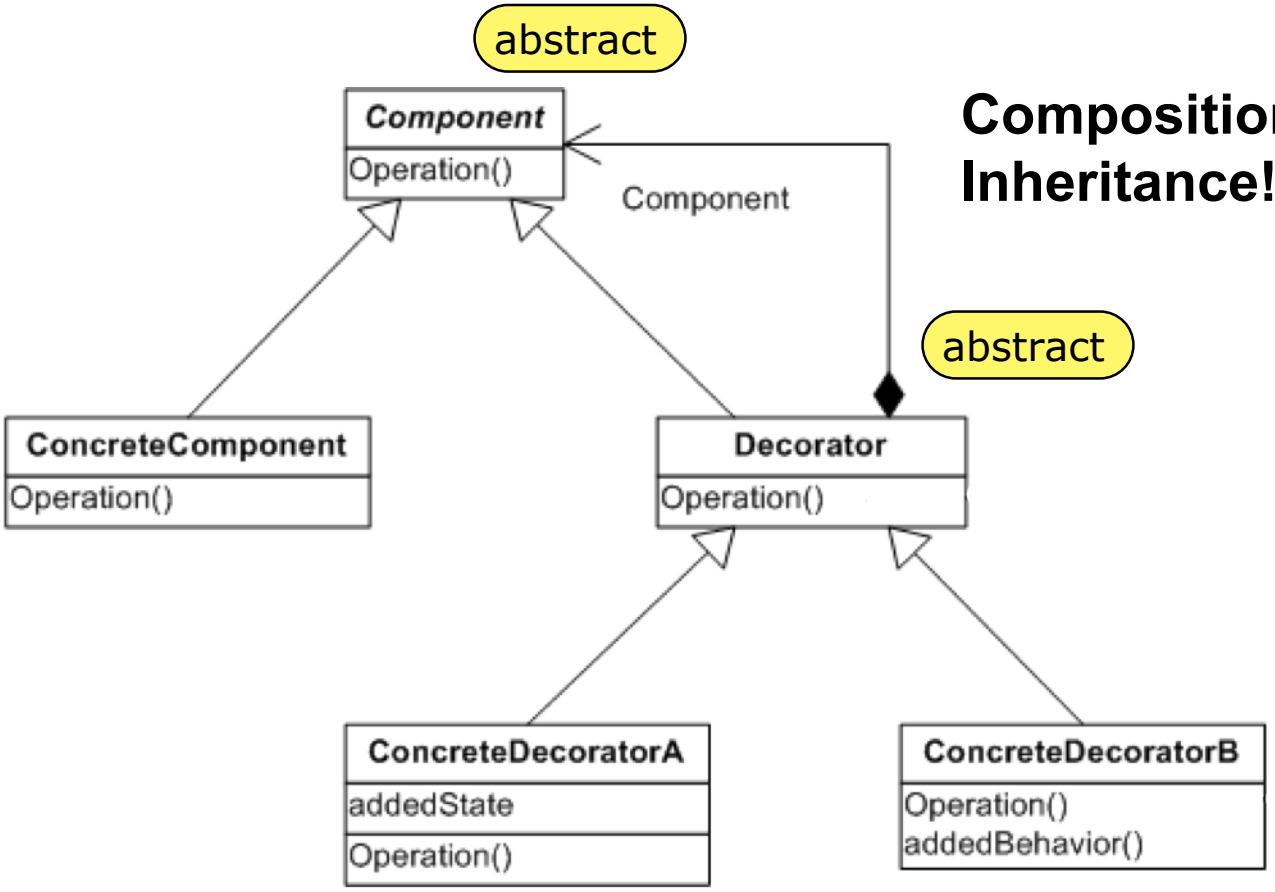
# Properties of Decorators

- We have Objects and Decorators
- Decorators have the same supertype as objects they decorate.
- You can pass around a decorated object instead of the original.
- The Decorator adds it's own behavior before or after delegating to the object it decorates
- Objects can be decorated dynamically at runtime.

# “Wrap” bagels with decorators



# Class diagram



# Decorator Class

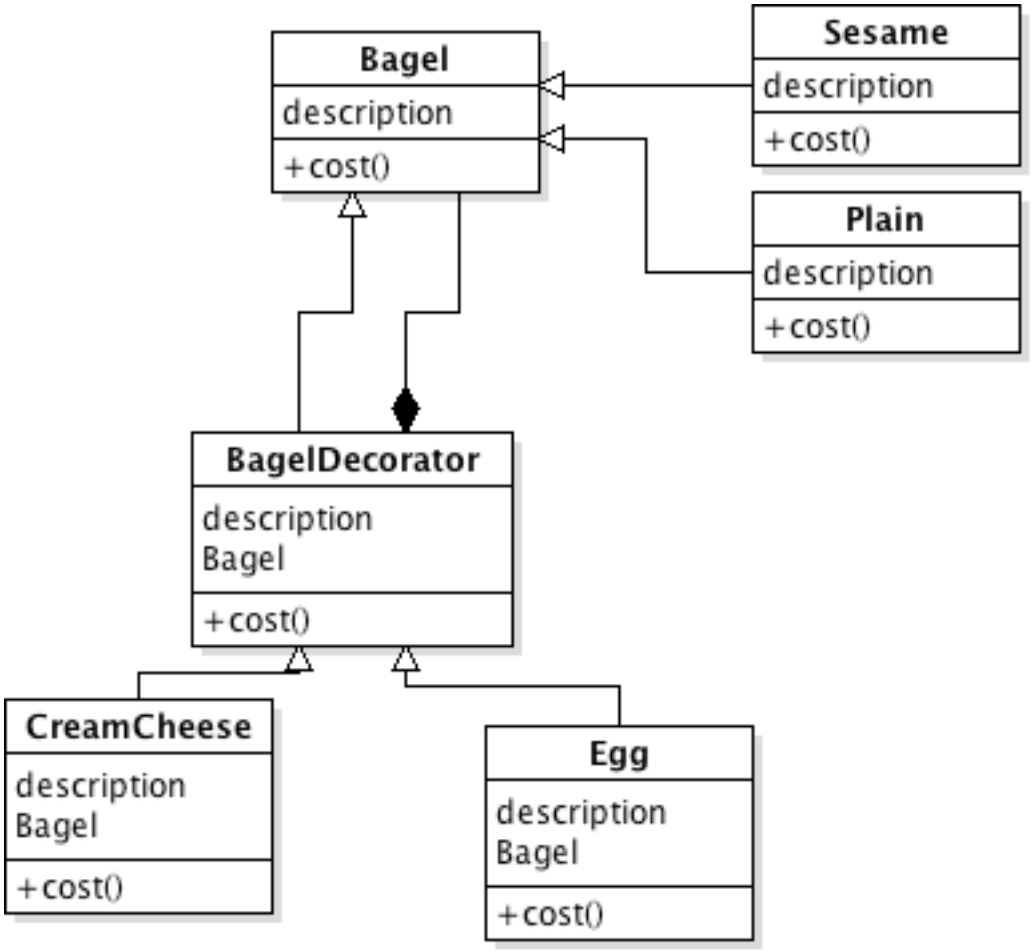
```
public class ConcreteDecoratorA extends Decorator {

    Component component;

    public ConcreteDecoratorA(Component component) {
        this.component = component;
    }

    public double methodA() {
        return component.methodA() + 3;
        // return 3 + component.methodA();
    }
}
```

instance variable holding a pointer to a component

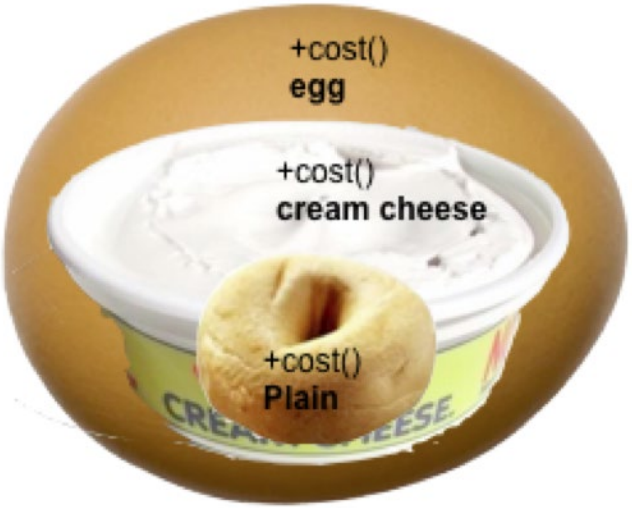
set this in constructor

add behavior before or after the function call

# Class Diagram



# Delegation to access behavior




1. egg.cost()
2. egg calls CC.cost()
3. CC calls plain.cost()
4. plain returns $plain
5. CC returns $plain + $cc
6. Egg returns $egg + ($plain+$cc)

# Bagel Class

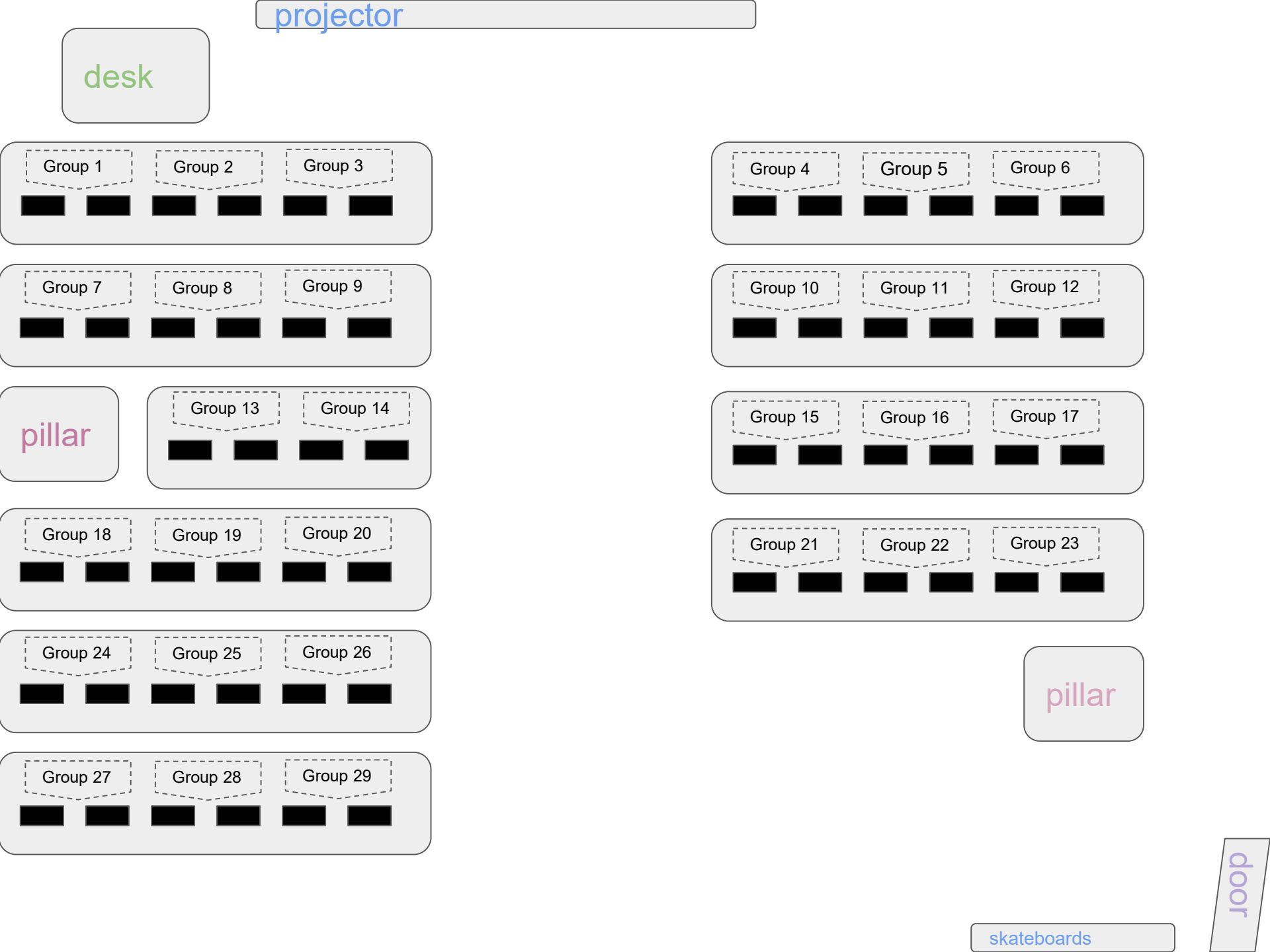
```
public abstract class Bagel {
    String description = "unknown Bagel";

    public String getDescription() {
        return description;
    }

    public abstract double cost();
}

public class Plain extends Bagel {
    public Plain() {
        description = "Plain";
    }

    public double cost() {
        return 1.00;
    }
}
```

# Bagel Decorator

```
public abstract class BagelDecorator extends Bagel {
    Bagel bagel;

    public abstract String getDescription();
}

public class Egg extends BagelDecorator {
    public Egg(Bagel bagel) {
        this.bagel = bagel;
    }

    public String getDescription() {
        return bagel.getDescription() + ", Egg";
    }

    public double cost() {
        return .50 + bagel.cost();
    }
}
```

# The main driver

```
public class BagelStore {
    public static void main(String[] args) {
        Bagel bagelSandwich = new Plain();
        bagelSandwich = new Creamcheese(bagelSandwich);
        bagelSandwich = new Egg(bagelSandwich);

        System.out.println("A bagel sandwich: " +
                                bagelSandwich.getDescription());

        System.out.println(" price: " + bagelSandwich.cost() +
                                " dollars\n");
    }
}
```


projector
desk
Group 1
Group 2
Group 3
Group 4
Group 5
Group 6
Group 7
Group 8
Group 9
Group 10
Group 11
Group 12
pillar
Group 13
Group 14
Group 15
Group 16
Group 17
Group 18
Group 19
Group 20
Group 21
Group 22
Group 23
Group 24
Group 25
Group 26
pillar
Group 27
Group 28
Group 29
door
skateboards